



LECTURE 4

DEEP REPRESENTATION LEARNING I

What is Learning?

PRESS – OR SPACE. →

© Marc Rußwurm

Licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#).

You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.

Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

Two Questions for Lectures 4–5

LECTURE 4

How does a neural network learn?

LECTURE 5

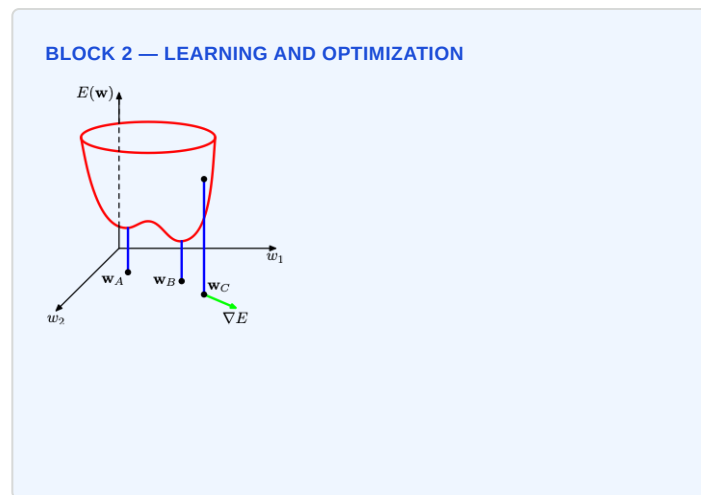
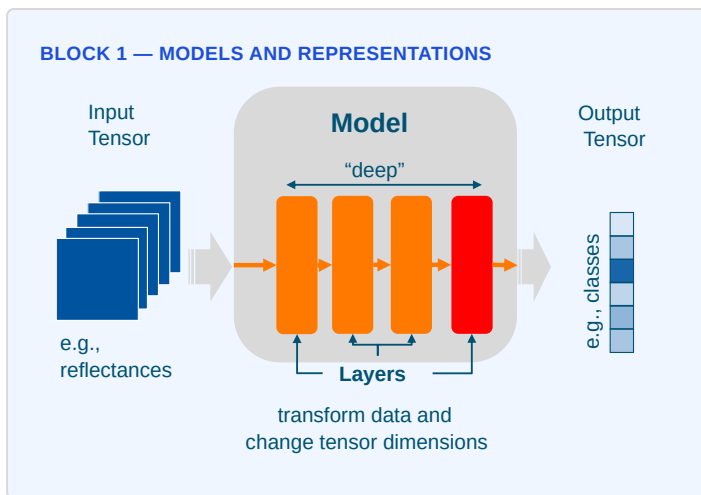
**Why should what it learns work
beyond the training data?**

Learning Outcomes Roadmap

Learning outcome	Lecture	Block
☐ Explain how neural networks learn representations that simplify tasks.	Lecture 4	Models and Representations
☐ Explain layers, parameters, biases, and nonlinear activations in an MLP.	Lecture 4	Models and Representations
☐ Explain forward passes, losses, gradients, backpropagation, and updates.	Lecture 4	Learning and Optimization
☐ Explain the inductive biases of CNNs, RNNs, GNNs, Transformers, and common losses.	Lab 4	Expert Jigsaw
☐ Distinguish finite datasets from samples of an underlying distribution.	Lecture 5	Samples and Distributions
☐ Explain generalization, overfitting, bias–variance, and train/validation/test evaluation.	Lecture 5	Classical Generalization
☐ Explain interpolation thresholds, double descent, and overparameterization.	Lecture 5	Modern Generalization

Today: How Do Neural Networks Learn Useful Representations?

Focusing on the first three outcomes from the roadmap.

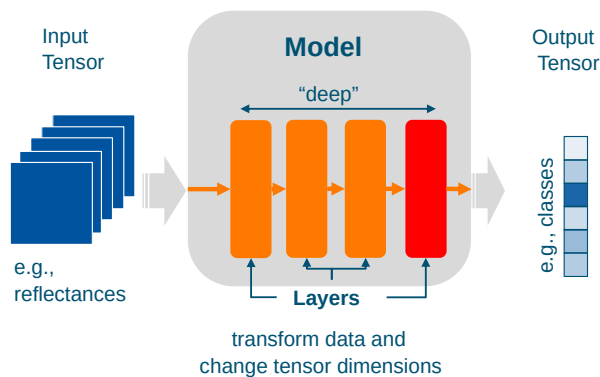


Next: what does learning actually mean for a system?

Two Core Topics in This Lecture

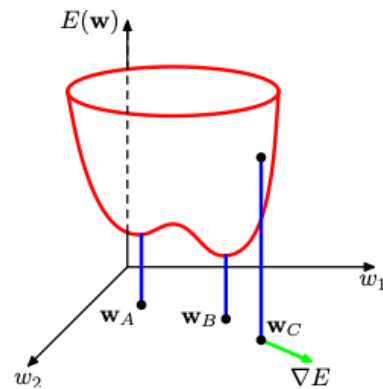
DEEP LEARNING MODEL

1. **Architectures** — MLP, CNN, Transformer
2. **Tasks** — Classification & Segmentation

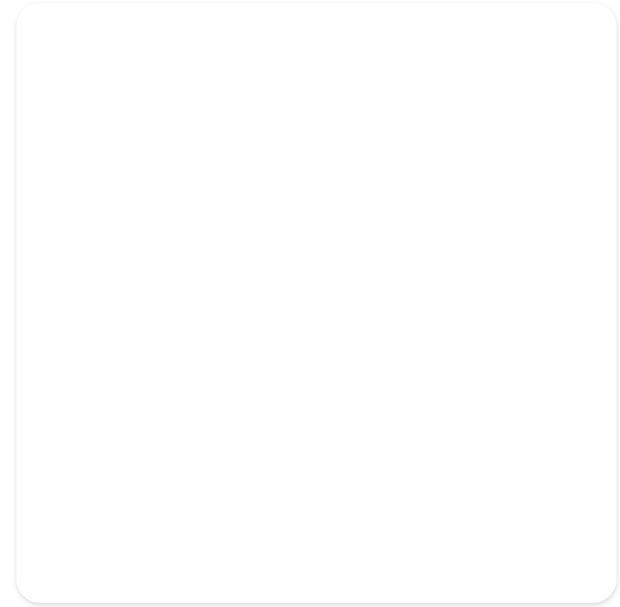


MODEL TRAINING

1. **Loss surfaces** and gradient descent
2. Forward pass → Loss → Backprop → Update

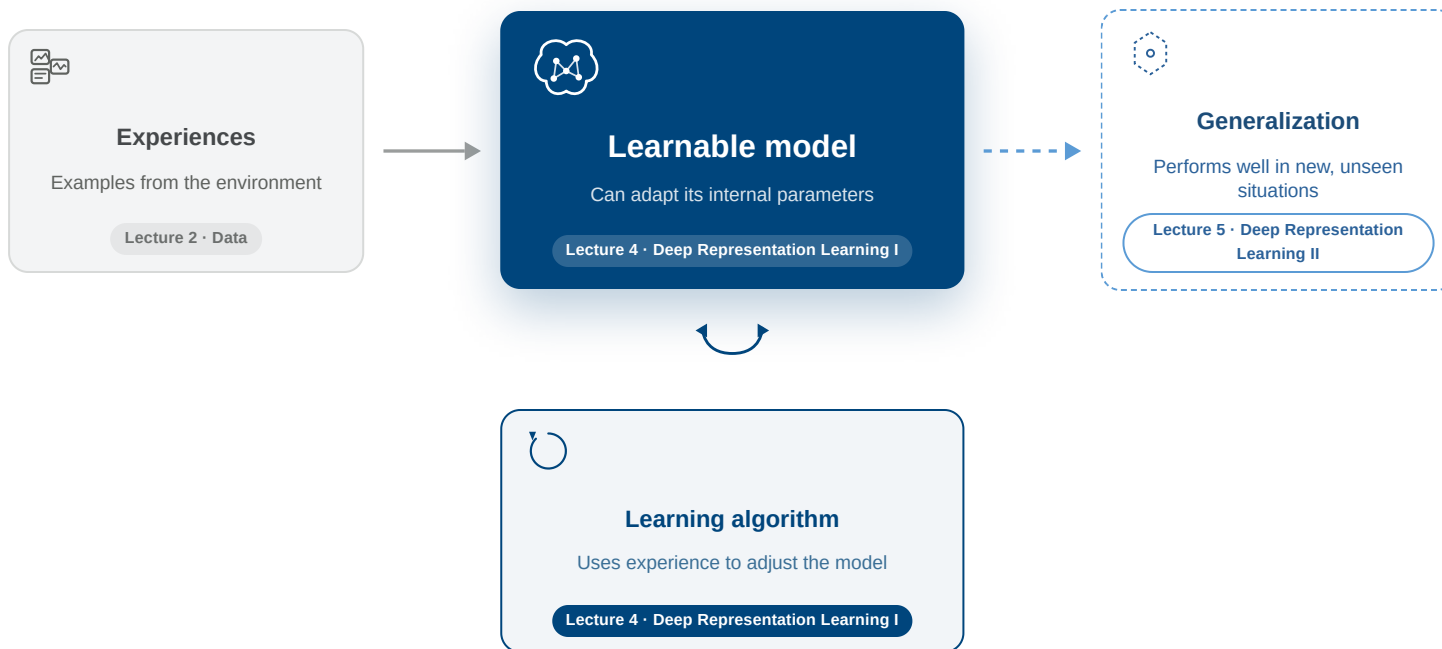


The Aspects of Learning



via [GIPHY](#), original source [imgur.com/gallery/FBMm2DO](#)

What Does a System Need to Learn?



INGREDIENTS FOR LEARNING

SUCCESS CRITERION

Timeline and Evolution of Deep Learning

<2012	2015 onwards	>2020 →
Feature Design: Which features extract the most information?	Model Design: Which architecture is best for my tensors?	Loss Design: Which loss function or training strategy?
Data: small labelled	Data: large labelled	Data: large unlabelled
Models: Random Forest, Boosting, Lin. Regression	Models: MLPs, CNNs, RNNs	Model: Transformers

Models and Representations

Our Learned Understanding

CHINESE

□□□□□□□□□□□□□□□□

ARABIC

لقد تعلّم عقلنا أن يفهم هذه الجملة.

HEBREW

המוח שלנו למד להבין את המשפט הזה.

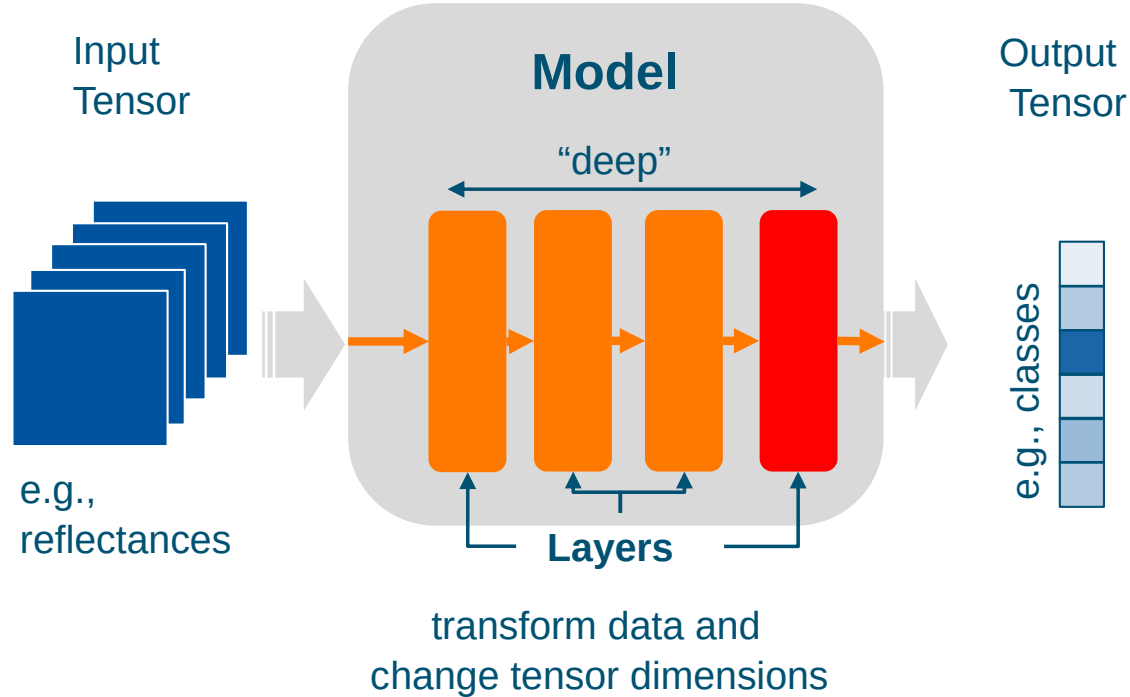
KOREAN

□□□ □□□ □ □□□ □□□□ □□ □□□□□.

LATIN

Our mind has learned to understand this sentence.

Deep Learning Model



Sentinel-2 as an Image Tensor

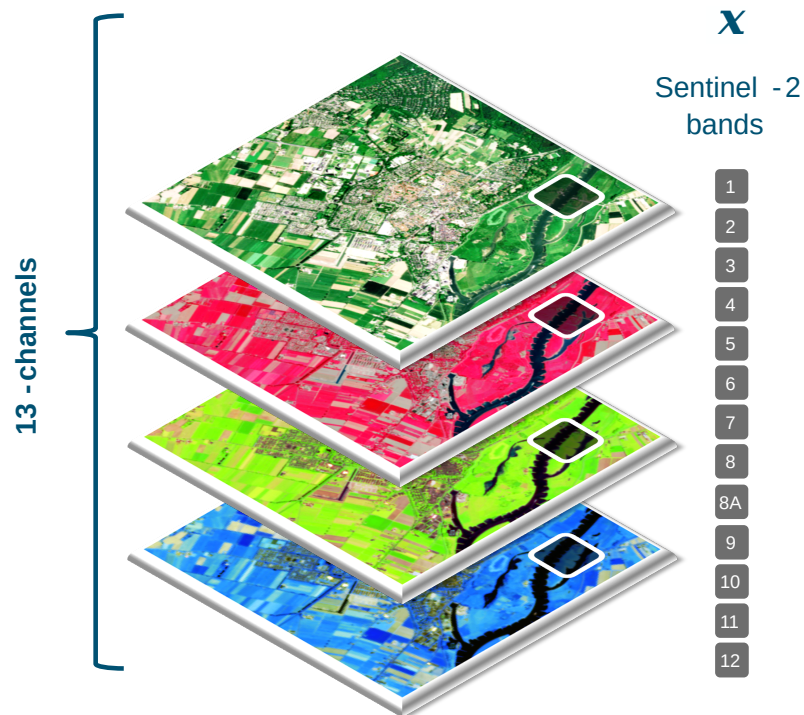


Image Classification

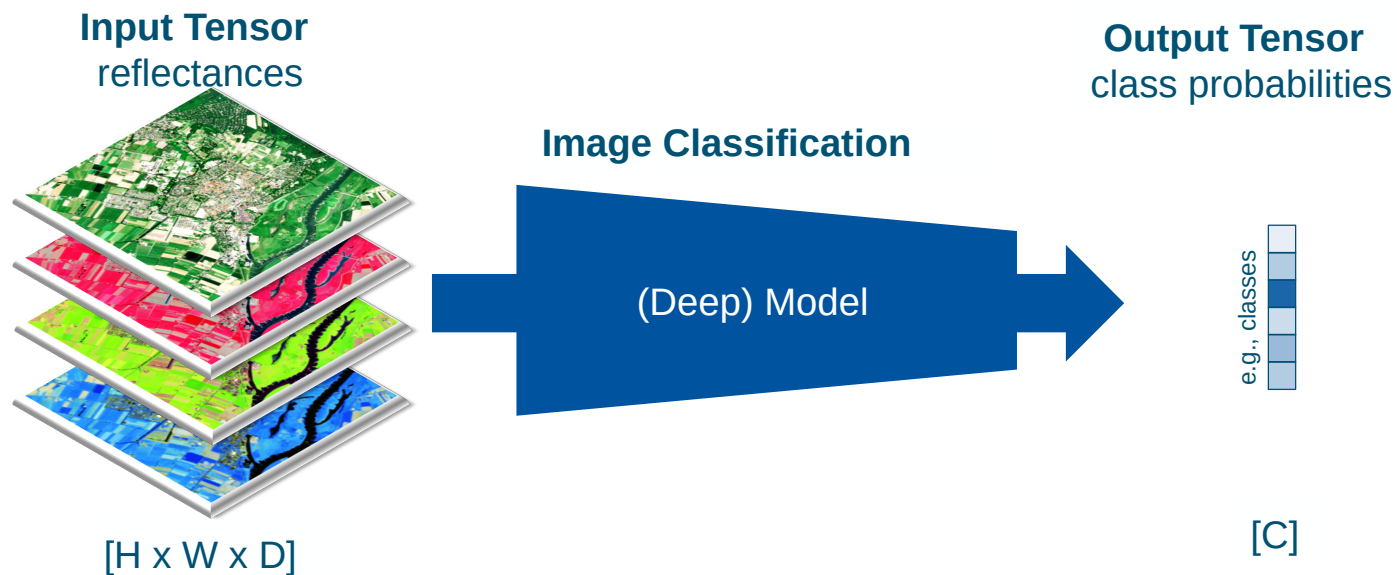
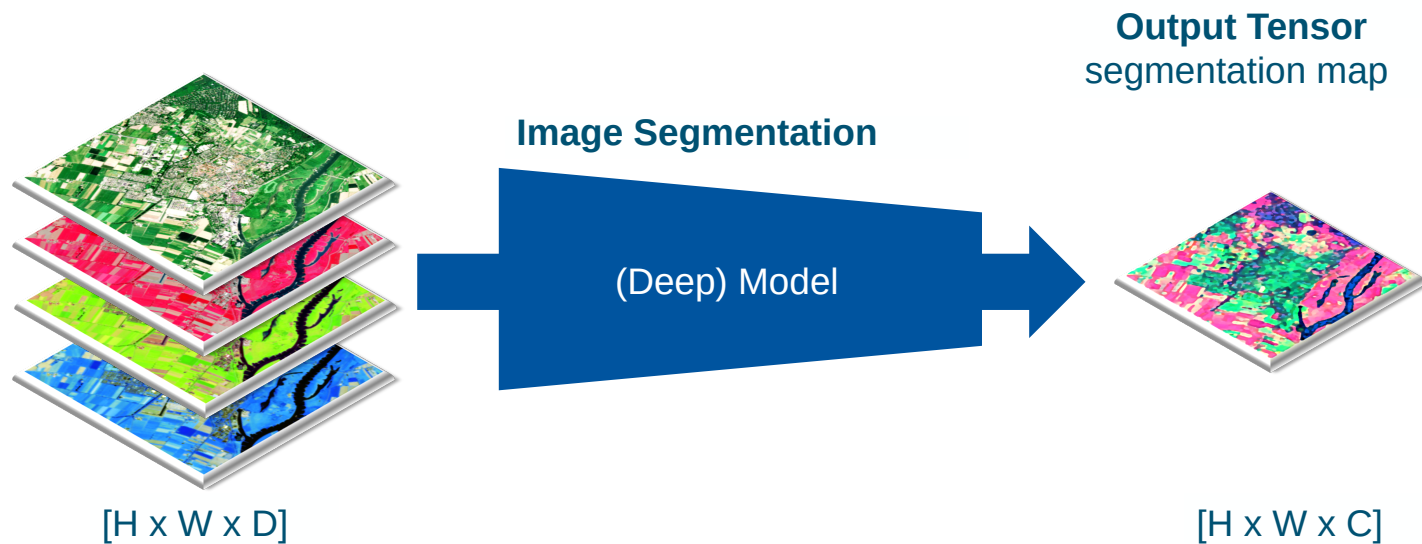


Image Segmentation



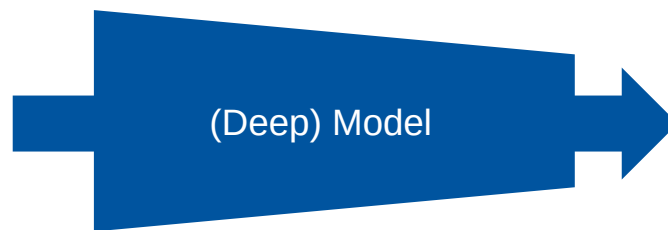
Object Detection

Input Tensor
reflectances



$[H \times W \times 3]$

Object Detection

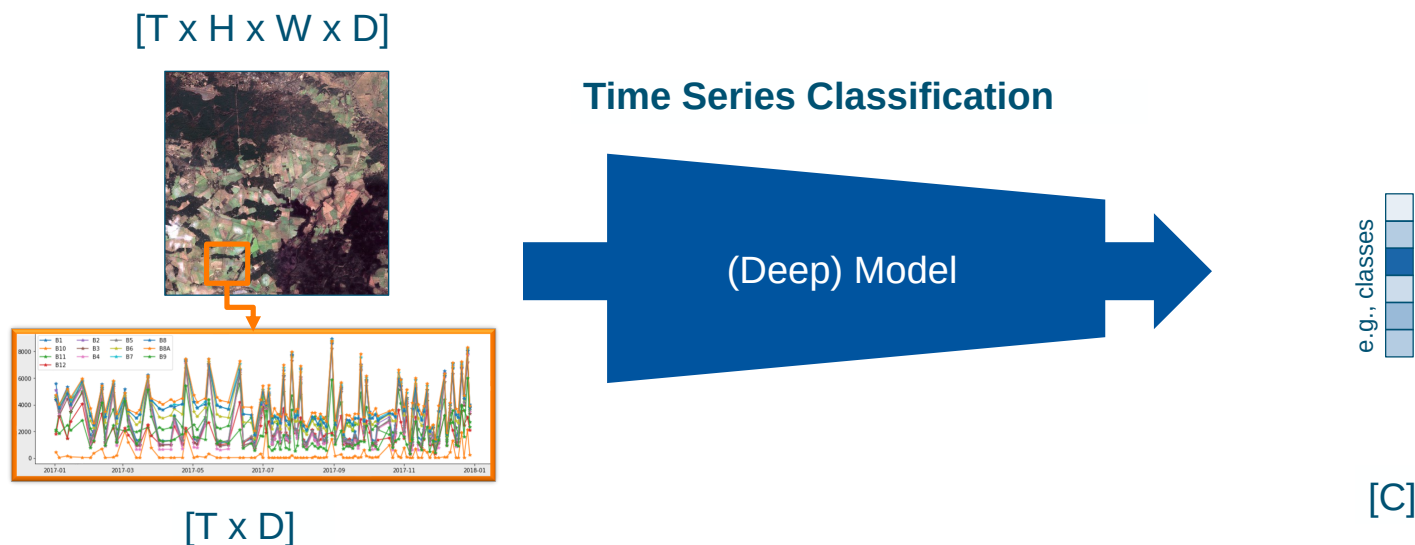


Output Tensor
bounding boxes

X	Y	H	W	
Green	Green	Green	Green	Bbox 1
Red	Red	Red	Red	Bbox 2
Yellow	Yellow	Yellow	Yellow	Bbox 3
Blue	Blue	Blue	Blue	...

$[N \times 4]$

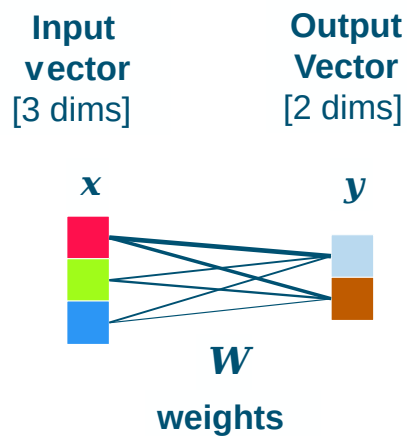
Time Series Classification



Overview: Deep Model Architectures

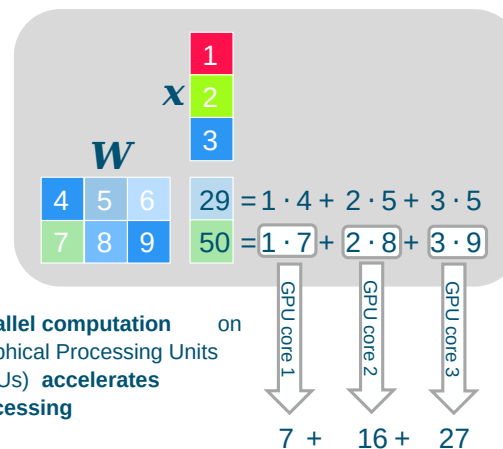
1950s	1990s	2015	2020s →
MLP ▪ Linear Projections ▪ Activation Functions ▪ Linear Algebra & Biological Analogy ▪ Parallelism on GPUs	CNN ▪ Convolutions ▪ Pooling ▪ Recep. Fields	ResNet ▪ Deeper Networks ▪ Batch Normalization ▪ Skip Connections ▪ Classification & Segmentation Tasks	Transformers ▪ Attention Mechanism ▪ Self-Attention (SA) ▪ Multi-Head SA ▪ Transformer Layers

Linear Transformation



Linear Transformation

$$y = Wx = \sum w_i x_i$$

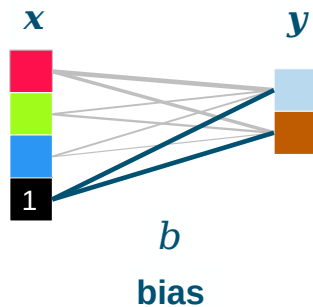


Parallel computation on
Graphical Processing Units
(GPUs) **accelerates**
processing

Linear Transformation with a Bias Term

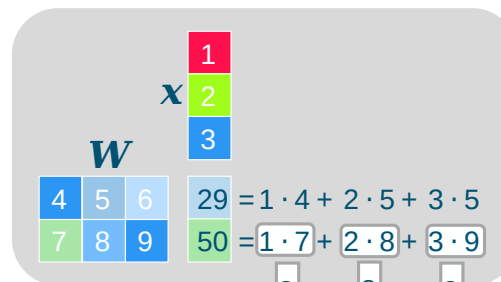
Input
vector
[3 dims]

Output
Vector
[2 dims]

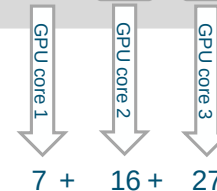


Linear Transformation

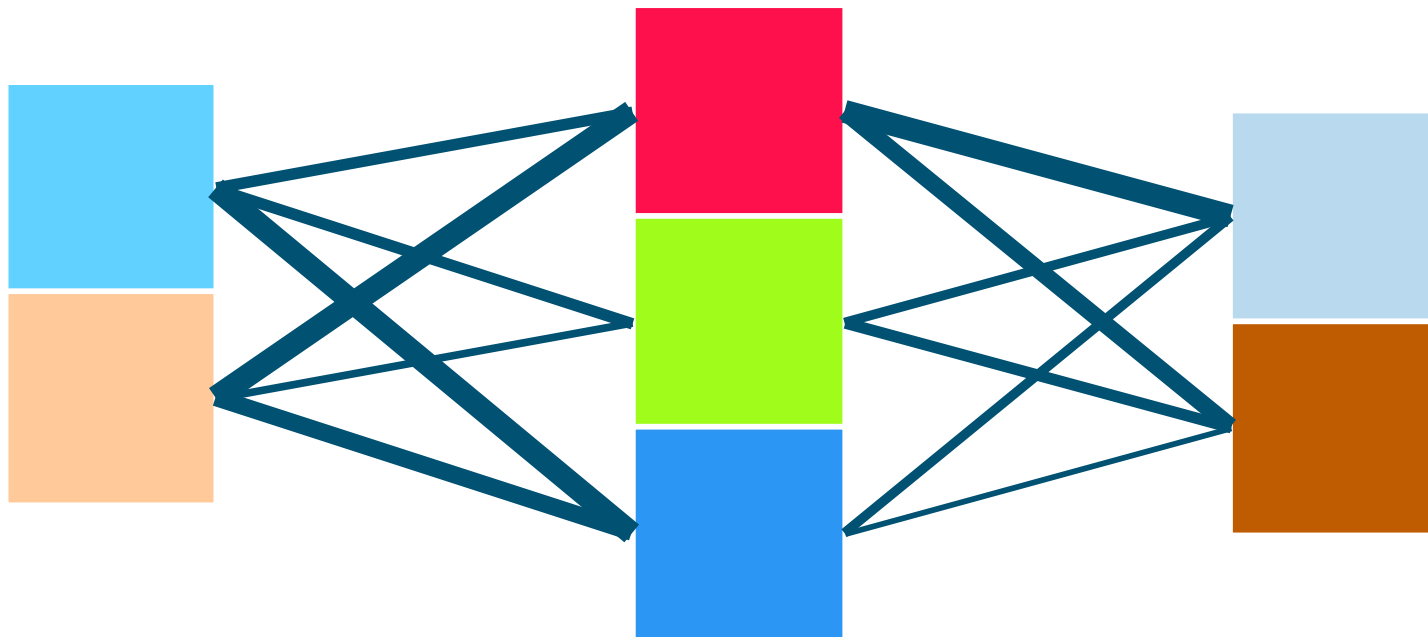
$$y = Wx = \sum w_i x_i + b$$



Parallel computation on
Graphical Processing Units
(GPUs) accelerates
processing

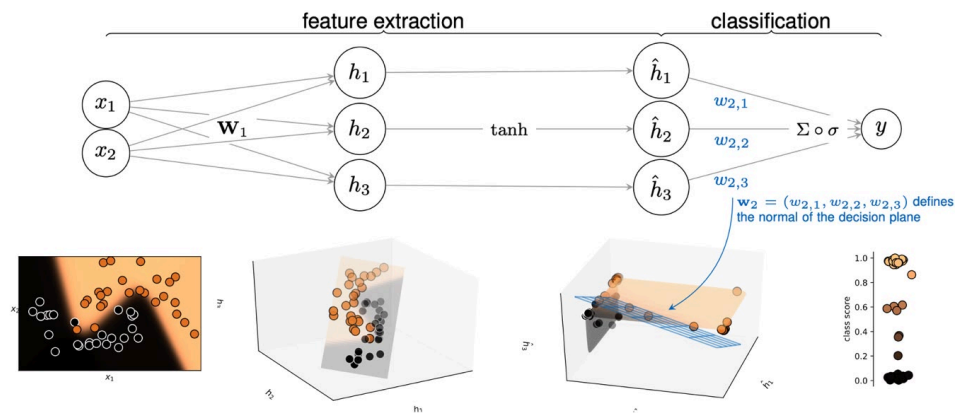


Weighted Connections



More "Deeper" Layers → more Complex Transformations

More "Deeper" Layers → more Complex Transformations



(a) Input space
 $\mathbf{x} = (x_1, x_2)$

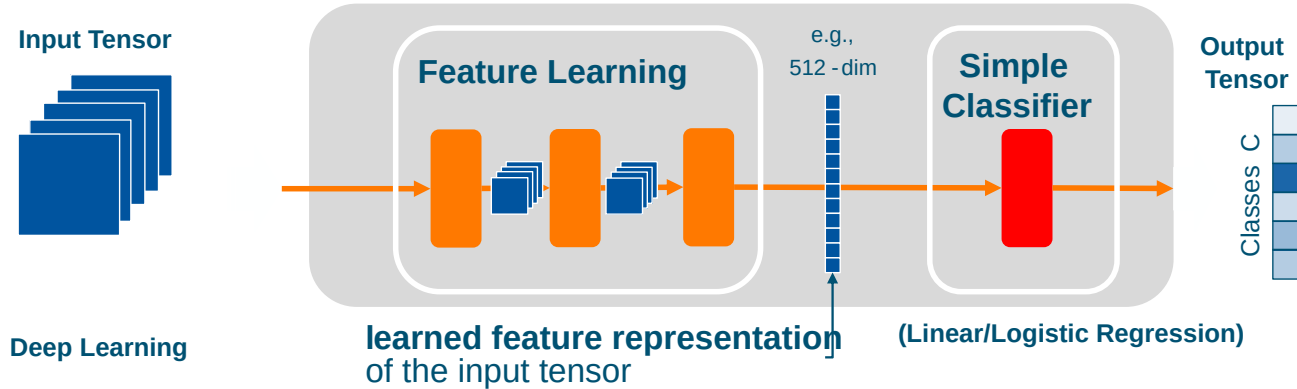
(b) linear projection
 $w_1^T \mathbf{x}$ into $\mathbb{R}^3$

(c) non-linear distortion
with $\tanh(w_1^T \mathbf{x})$

(d) Output class probabilities
 $y \in [0, 1]$

 Loading slide...

Machine Learning and Deep Learning



Machine Learning

Input Tensor



Design knowledge in **designing** features mathematically

Features Design

(e.g., NDVI, NDWI, GLCM Textures)

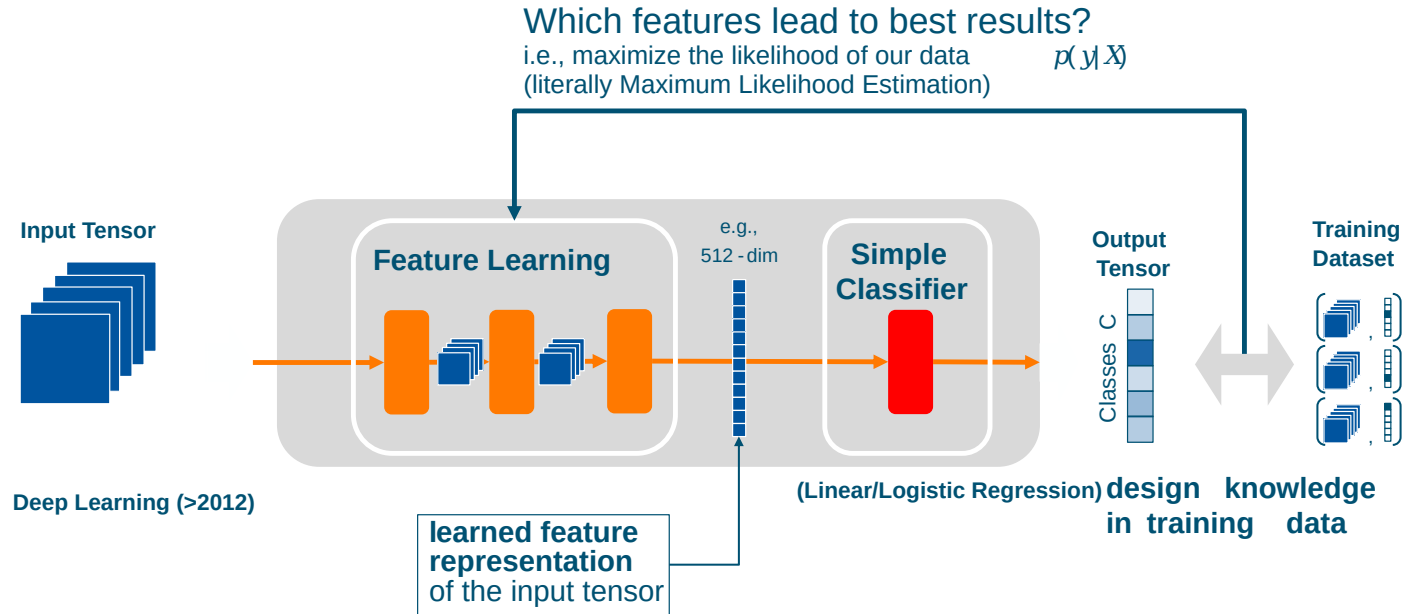
Complex Classifier

(Random Forest, Boosting)

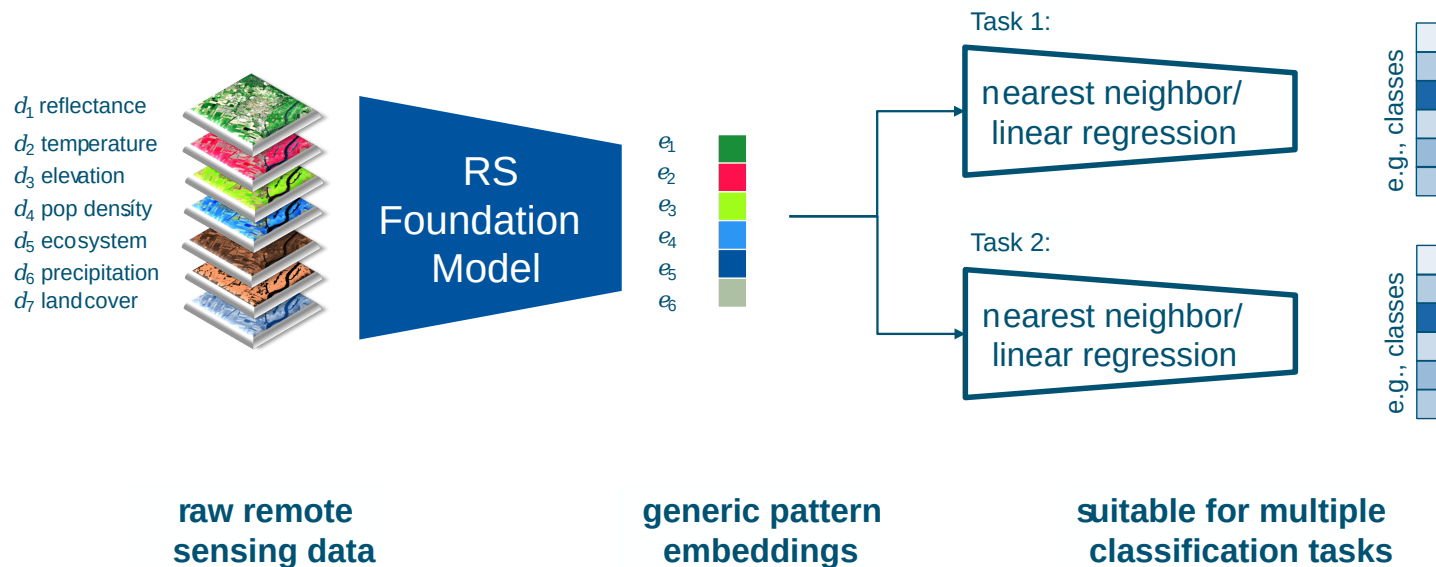
Output Tensor



Feature Learning



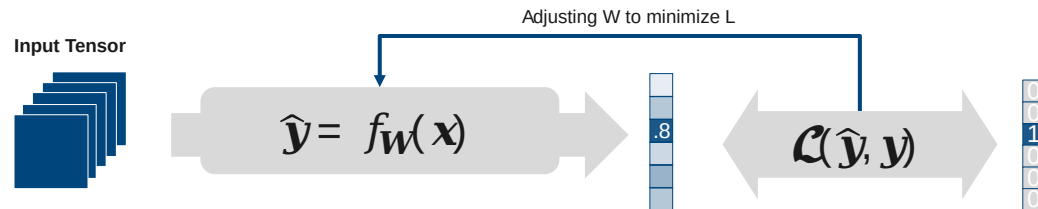
Foundation Models



From predictions to loss, gradients, and parameter updates.

Learning and Optimization

Training: Adjusting Weights to Minimize Loss



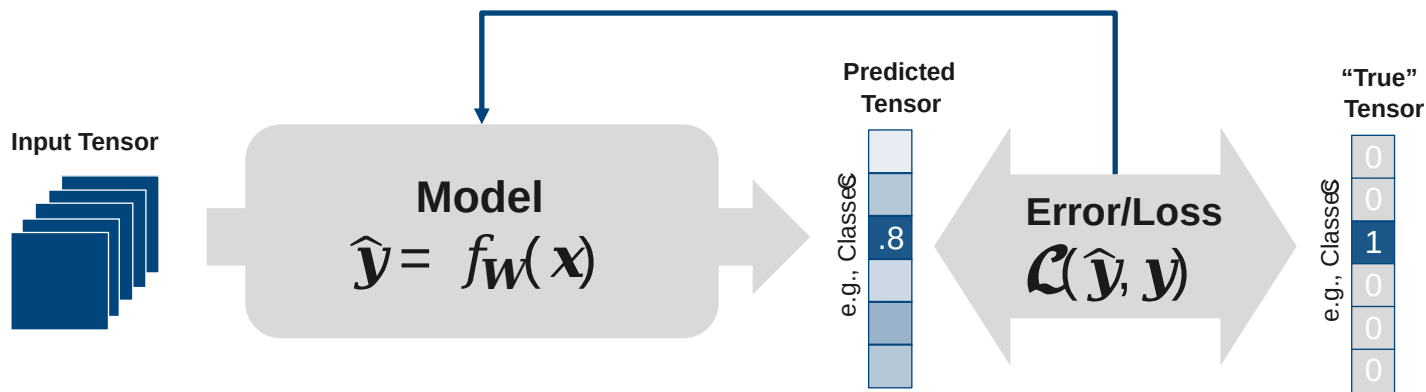
$$\mathcal{L}(\mathcal{D}_{\text{train}}) = \sum_{\mathbf{x}, \mathbf{y} \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



The Learning Objective: argmin

Minimize Error/Loss function

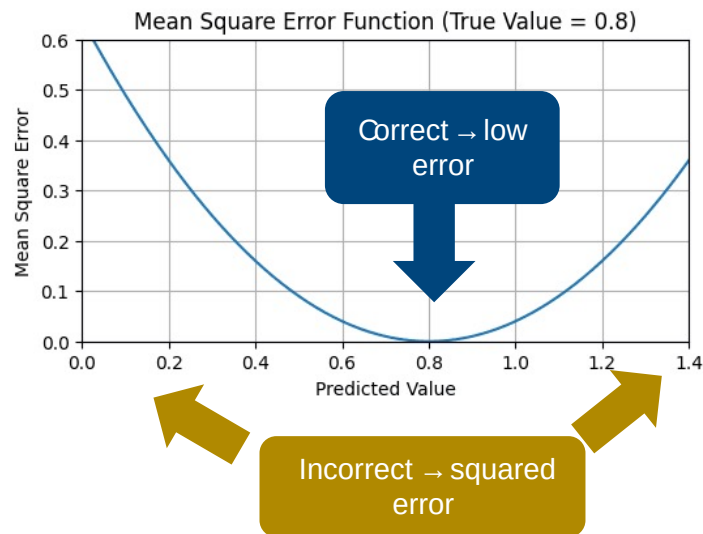
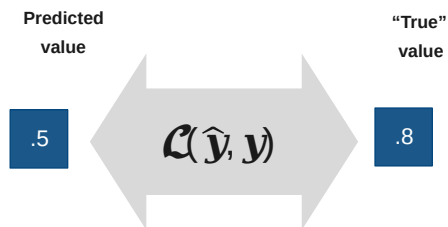
$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f(\mathbf{x}), \mathbf{y})$$



Loss Function: Mean Squared Error

Regression: Mean Squared Error

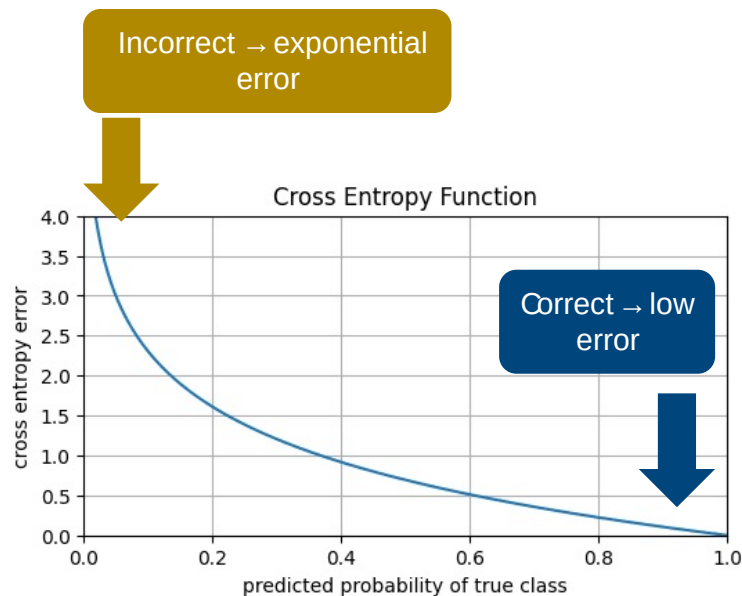
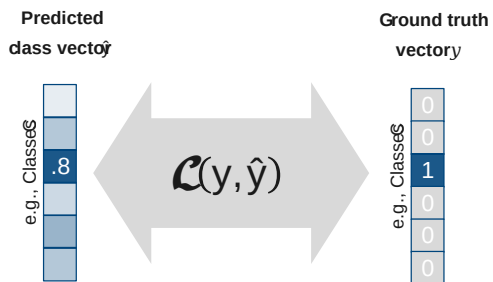
$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$



Loss Function: Cross-Entropy

Classification: Cross Entropy

$$\mathcal{L}(y, \hat{y}) = \sum_{c \in \text{Classes}} -y_c \log(\hat{y}_c)$$



Loss Surfaces

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

more complex models have more complex loss surfaces

Gradient Descent

Minimize Error/Loss function

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \mathcal{L}(f_{\mathbf{W}}(\mathbf{x}), \mathbf{y})$$

Through Iterative Gradient Descent:

$$\mathbf{W}^{(i+1)} = \mathbf{W}^i - \nabla \mathcal{L}(f_{\mathbf{W}}, \mathcal{D}_{\text{train}})$$

new weights

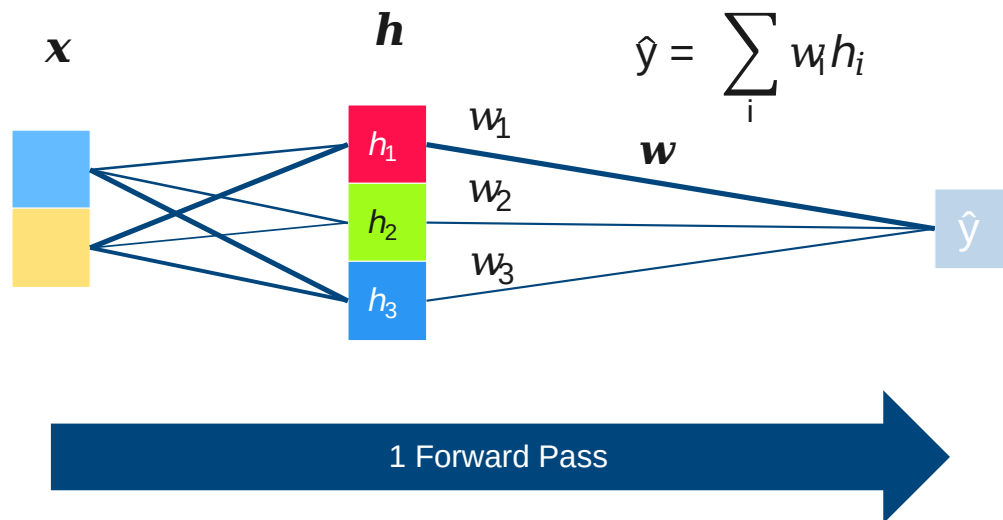
old weights



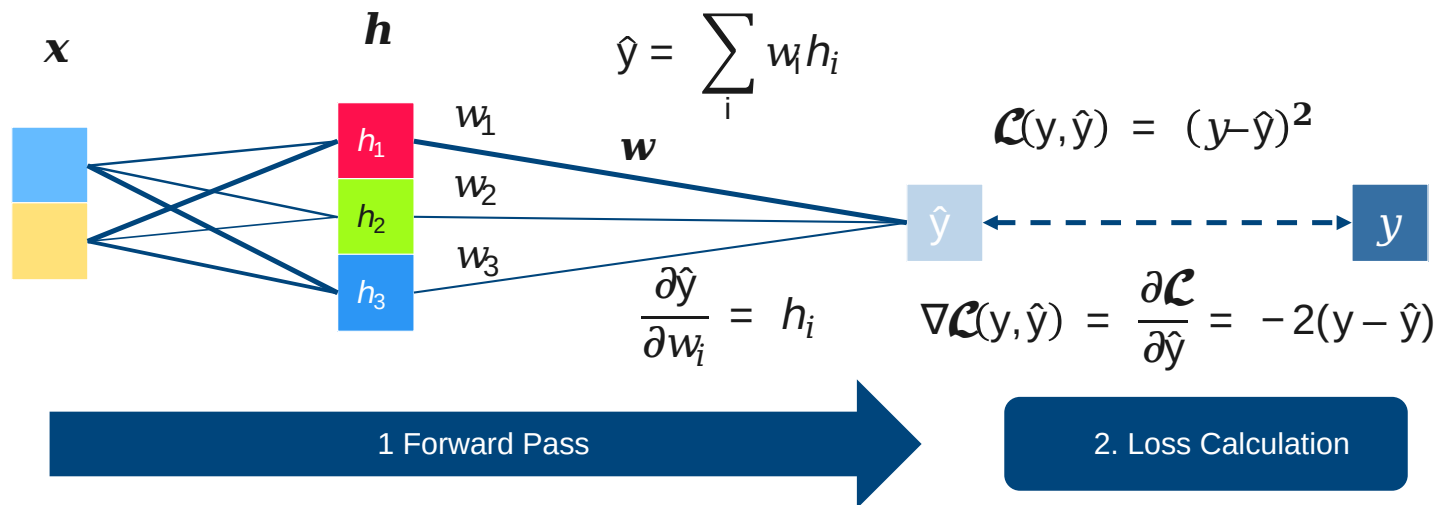
Gradients of the Loss function over a “batch” of training data $\mathcal{D}_{\text{train}}$

“What weights change the loss the most?”

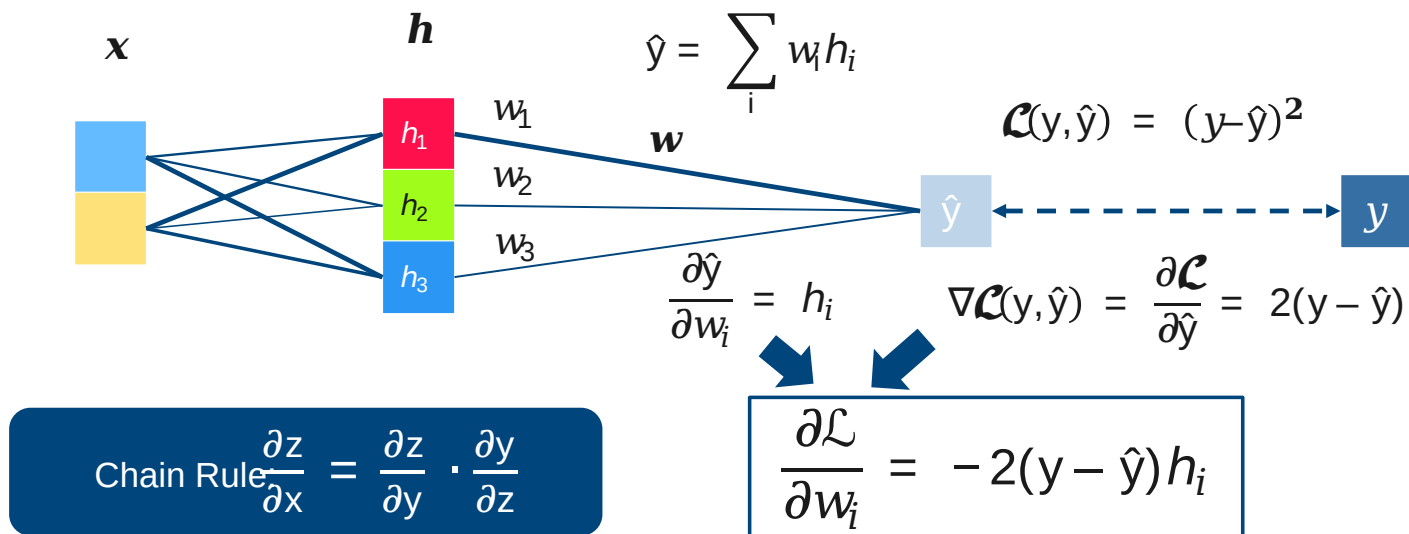
Backpropagation



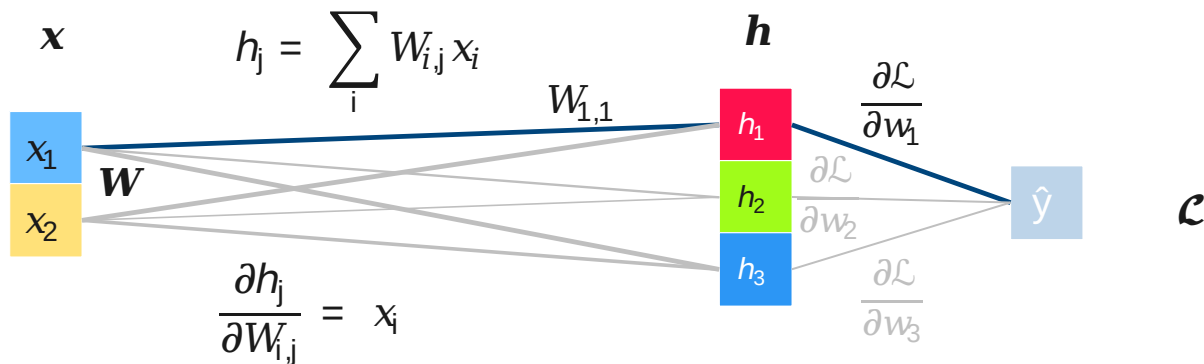
Backpropagation



Backpropagation

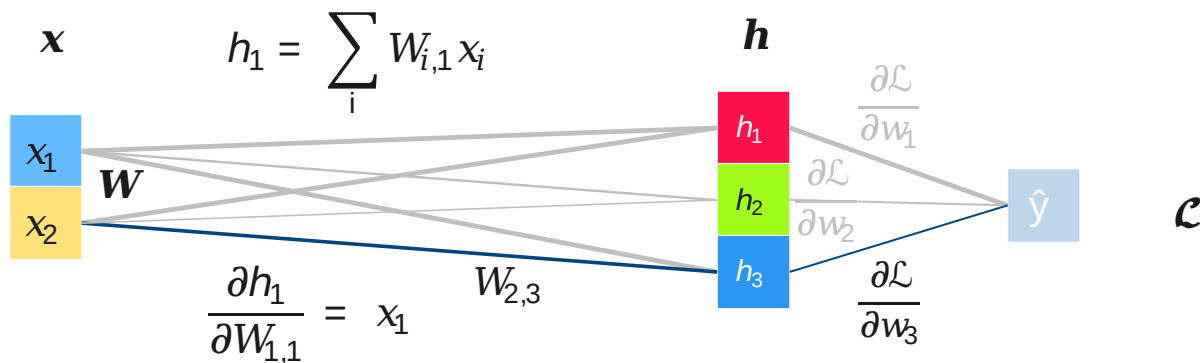


Backpropagation



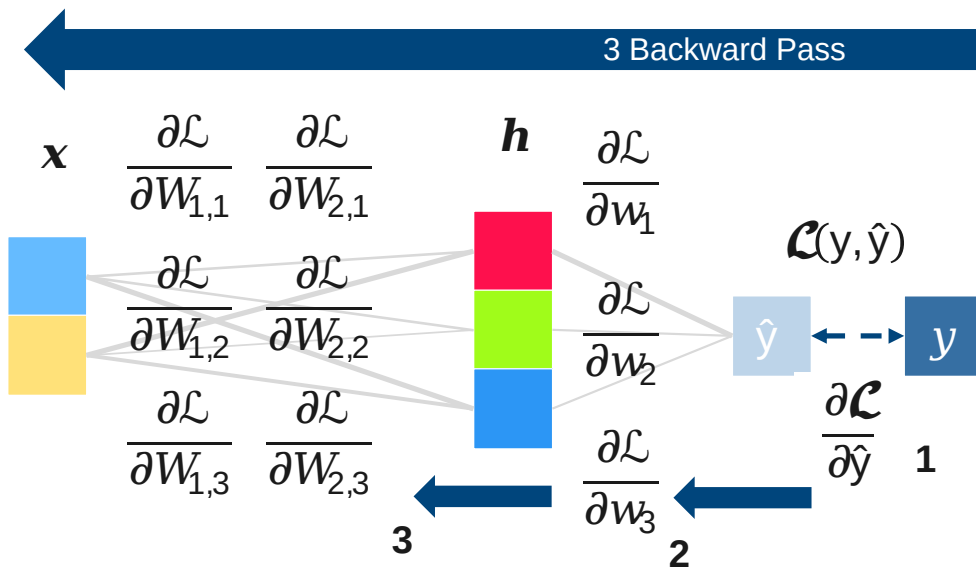
$$\frac{\partial \mathcal{L}}{\partial W_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1} = -x_1 2(y - \hat{y}) h_1$$

Backpropagation



$$\frac{\partial \mathcal{L}}{\partial W_{2,3}} = x_2 \frac{\partial \mathcal{L}}{\partial w_3} = -x_2 2(y - \hat{y}) h_3$$

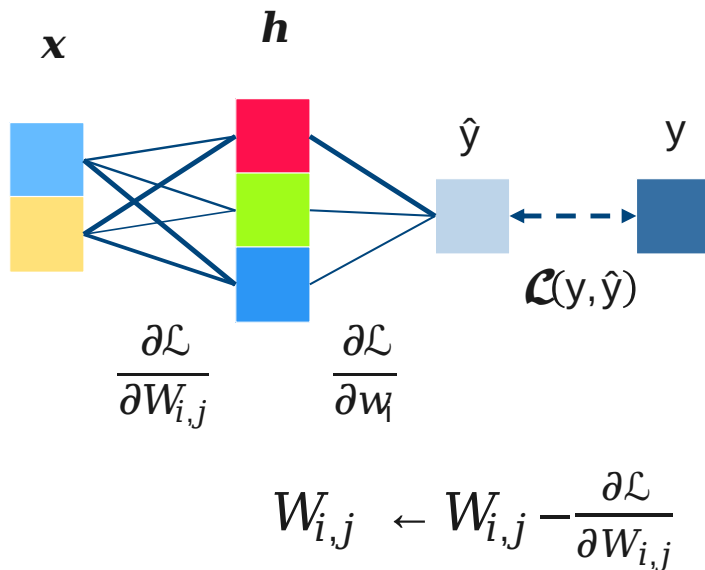
Backpropagation



Save computation:
Gradients of earlier
layers require only
Gradients from the
pervious layer

$$\frac{\partial \mathcal{L}}{\partial w_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1}$$

Backpropagation



Forward Pass

```
y_pred = model(x)
```

Loss Computation

```
loss = MSE(y_pred, y)
```

Backward Pass

```
loss.backward()
```

Update Gradients

```
optimizer.step()
```